

Response to Reviewer Feedback

Ecosystem Simulation — Point-to-Point Rebuttal

Name: Yermek Aubayev

Matricula Number: 551098

Course: Data Analysis

Professor: Salvatore Distefano

University of Messina

Each of the four remarks below is quoted verbatim, followed by what was found to be accurate in it, exactly what changed in the code, and how the change was verified. Every code change described here was applied to the actual source, compiled, and checked against the full test suite (107 tests, 0 failures) before this document was written.

1. “Encapsulation and information hiding are two different concepts, now they are overlapped and mixed up in encapsulation, some concepts of the latter are used in the former generating confusion.”

What was actually wrong. The previous §2.1 (Encapsulation) description read: “*Every field on Entity/Organism/Animal follows the same pattern: protected state, exposed only through explicit getters/setters.*” That sentence is an information-hiding claim (about field visibility across an entire class hierarchy), placed inside the encapsulation section — exactly the mixing described above.

Fix.

- §2.1 (Encapsulation) is now scoped strictly to *access control on one object’s own state*: `World.getEntities()` returning a defensive copy so a caller cannot mutate `world`’s internal list behind its back. This is a claim about one method’s contract, nothing about field visibility across a hierarchy.

- §2.2 (Information Hiding) is now scoped strictly to *whether an attribute's representation is free to change later without breaking other code*, and no longer repeats the encapsulation example.
- A new closing paragraph in §2.1, “Encapsulation versus information hiding, made explicit,” states the distinction directly so the two cannot be conflated again on a re-read.

2. “Information hiding is not well enforced, there are non-private attributes despite your claims. Information hiding should enforce the 3-step information-hiding engineering (private, class or instance, constant or variable).”

This was correct. The following fields were protected, not private, despite the report's accessor-based claims:

File	Fields that were protected
Entity.java	id, x, y, alive, world, statistics, simulationEventListener, simulationTime, schedulingContext
Organism.java	energy, age, maxEnergy
Animal.java	speed, visionRange
Predator.java	attackPower
Herbivore.java	defensePower

protected in Java is reachable from any subclass **and** from any class in the same package — public getters existed for all of these, but nothing stopped a subclass from bypassing them, and several actually did: e.g. `Predator.hunt()` read the inherited `visionRange` field directly (`world.getNeighbors(this, visionRange)`) instead of calling `getVisionRange()`, and `Predator/Herbivore/Plant` all read `world`, `schedulingContext`, and (in `Predator`) `statistics` as bare inherited fields rather than through `getWorld()/getSchedulingContext()/getStatistics()`.

Fix (code).

- Every field listed above is now `private`. `Entity.id` is now also `final` — a private, immutable field is a compiler-enforced constant, not just a documented one.
- ~20 call sites in `Predator.java`, `Herbivore.java`, and `Plant.java` (inside `hunt()`, `attack()`, `reproduce()`, `graze()`, and both constructors) were rewritten to go through the existing accessor methods instead of the field directly.
- The report's §2.2 now applies the exact 3-step method to three concrete fields, covering all three combinations your slides distinguish:

Attribute	1. Private?	2. Scope	3. Constant/ variable
<code>Entity.id</code>	Yes	Instance	Constant (<code>final</code>)
<code>Entity.nextId</code>	Yes	Class (<code>static</code>)	Variable
<code>Organism.energy</code>	Yes	Instance	Variable

Verification. `mvn test` after the change: **Tests run: 107, Failures: 0, Errors: 0** — the fix changes only the access *path* (getter vs. bare field), not behavior, and every existing test (including ones exercising `Predator/Herbivore/Plant` directly) still passes unmodified.

3. “Subtyping-multityping should be shown by an example from the client program where you demonstrate a multitype object can be accessed by different types of variables.”

Fix (code). Added `Main.demonstrateMultityping()` to `Main.java` — the actual client entry point, executed at the start of every run, before the GUI opens. It constructs exactly **one** `Predator` object and assigns that same reference to five variables of five different declared types, calling a type-specific method through each:

```
// Main.java
Predator predator = new Predator(0, 0, 80, 2.0, 7);

Entity entityRef = predator;           // by inheritance
Organism organismRef = predator;      // by inheritance
Animal animalRef = predator;          // by inheritance
```

```

Movable movableRef = predator;           // by interface (Animal
implements Movable)
Reproducible reproducibleRef = predator;   // by interface (Predator
implements Reproducible)

System.out.println("as Entity           -> id=" + entityRef.getId() + ",
alive=" + entityRef.isAlive());
System.out.println("as Organism         -> energy=" +
organismRef.getEnergy());
System.out.println("as Animal           -> speed=" +
animalRef.getSpeed());
System.out.println("as Movable          -> getSpeed() via interface=" +
movableRef.getSpeed());
System.out.println("as Reproducible -> " +
reproducibleRef.getClass().getSimpleName() + " implements
reproduce()");
System.out.println("identity check -> entityRef == animalRef: " +
(entityRef == animalRef));

```

Actual console output when the program runs:

```

as Entity           -> id=1, alive=true
as Organism         -> energy=80
as Animal           -> speed=2.0
as Movable          -> getSpeed() via interface=2.0
as Reproducible     -> Predator implements reproduce()
identity check      -> entityRef == animalRef: true

```

The last line is the proof: `entityRef == animalRef` compares object identity, not declared type, and it is `true` — five variables of five different types, one underlying object. Report §2.7 now quotes this exact code and output, in addition to (not instead of) the existing type-hierarchy and `RuleVocabulary` discussion.

4. “The exception handling example is very weak, you need to handle an exception, not just define new ones.”

This was correct about the report. The previous §2.8 quoted only the two exception *class definitions* (constructors) and described handling in prose, without ever quoting an actual `catch` block. Real handling did already exist in the application (`RuleEditorPanel.doReload()`), but the report never showed it — and separately, the application’s startup path (`MainWindow`) turned out to have a genuinely weak catch: it

only wrote to `System.err`, which is invisible in a packaged Swing GUI and does not change the program's visible behavior at all.

Fix (report). §2.8 now quotes and explains two real, different catch blocks:

1. `RuleEditorPanel.doReload()` — catches `RuleParseException` and explicitly keeps the previous in-memory rules rather than letting a corrupted on-disk file reach the running engine. This is independently verified by `RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules`, not just asserted in prose.
2. `MainWindow`'s rule-loading path (constructor and `resetSimulation()`).

Fix (code). Both `MainWindow` catch blocks were changed from a console-only warning to a call into the application's own visible event log:

```
// MainWindow.java — before
} catch (IOException | RuleParseException e) {
    System.err.println("Warning: could not load active rules: " +
e.getMessage());
}

// MainWindow.java — after
} catch (IOException | RuleParseException e) {
    eventLogPanel.logEvent("Could not load config/rules.txt (" +
e.getMessage()
    + ") -- starting with no active rules. Use the Rule Editor
to add some.");
}
```

This leaves the program in one specific, well-defined state (an explicitly empty rule set) and tells the user, in the same on-screen log the running simulation already uses for every other event — real recovery, not a log line nobody running the packaged application would ever see.

Summary of all files touched: `Entity.java`, `Organism.java`, `Animal.java`, `Predator.java`, `Herbivore.java`, `Plant.java`, `Main.java`, `MainWindow.java`, plus `docs/report` (`final_report.md` regenerated to `.docx/.pdf`). No test file required modification; the full suite (107 tests) passes unchanged after every code fix above.